

Módulo de Bases de datos vectoriales

Entrega final

Diseño de Sistema RAG

1. Selección del modelo de embeddings

Para representar la información del documento se ha utilizado el modelo de embeddings **Gemini (models/gemini-embedding-001)** a través de la librería `google-generativeai`.

Este modelo permite convertir el texto en vectores numéricos que representan el significado del contenido. Gracias a estos vectores es posible buscar información similar dentro de la base de datos vectorial.

Se eligió este modelo porque ofrece buenos resultados en tareas de búsqueda semántica y además se integra fácilmente con el modelo de lenguaje Gemini utilizado para generar las respuestas.

En este proyecto no se ha realizado fine-tuning del modelo, aunque en un sistema real se podría ajustar con datos del dominio (por ejemplo noticias deportivas) para mejorar la recuperación de información.

2. Base de datos vectorial

Para almacenar los embeddings se ha utilizado **ChromaDB**.

Chroma es una base de datos vectorial open source que permite guardar vectores, texto y metadatos, y realizar búsquedas por similitud de forma sencilla desde Python.

En este sistema Chroma se ejecuta dentro de un **contenedor Docker**, lo que facilita su despliegue y separación del resto del sistema.

Cada fragmento almacenado contiene:

- el texto del fragmento
- su embedding
- metadatos como la página del documento o el origen

Estos metadatos ayudan a entender de dónde viene la información recuperada.

3. Estrategia para mantener la información actualizada

La información se introduce en la base vectorial mediante un proceso de **ingesta** implementado en el archivo `ingest.py`.

Este script realiza los siguientes pasos:

1. Cargar el documento PDF con noticias deportivas.
2. Dividir el texto en fragmentos más pequeños mediante **chunking**.
3. Generar un embedding para cada fragmento.
4. Guardar los fragmentos y sus embeddings en ChromaDB.

Para dividir el texto se ha utilizado `RecursiveCharacterTextSplitter`, con un tamaño de fragmento de aproximadamente **800 caracteres** y un pequeño solapamiento para mantener el contexto.

Este proceso permite trabajar con documentos grandes sin necesidad de enviar todo el texto al modelo.

4. Integración con el modelo de lenguaje (LLM)

El sistema utiliza un modelo de lenguaje de **Gemini** para generar las respuestas.

El funcionamiento del sistema sigue el enfoque **RAG (Retrieval Augmented Generation)**:

1. El usuario realiza una pregunta desde la interfaz de **Streamlit**.
2. La pregunta se convierte en un embedding.
3. ChromaDB busca los fragmentos más similares al significado de la pregunta.
4. Se recuperan los fragmentos más relevantes (por ejemplo los 3 mejores).
5. Estos fragmentos se envían al modelo de lenguaje como contexto.
6. El modelo genera una respuesta basada en ese contexto.

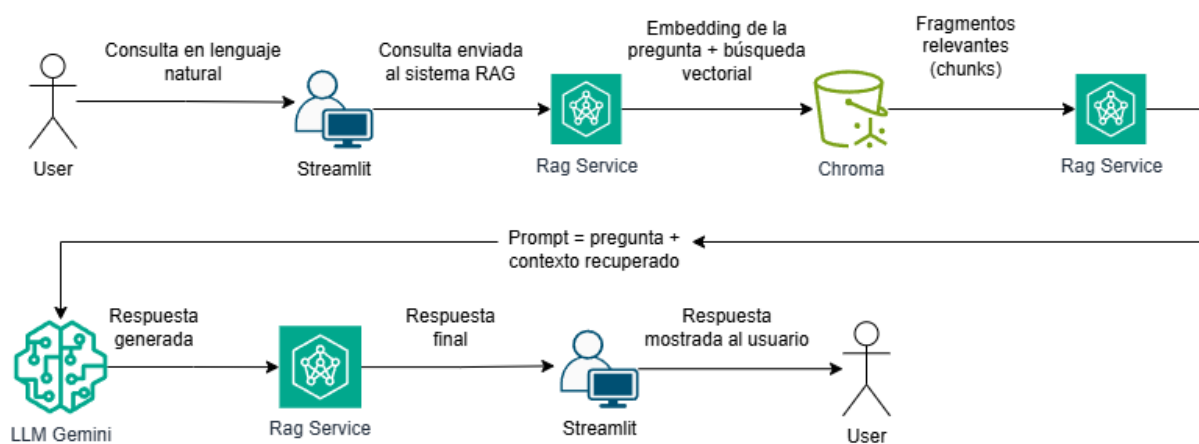
De esta forma el modelo responde utilizando información real del documento y se reduce el riesgo de generar respuestas incorrectas.

5. Arquitectura del sistema

El sistema está compuesto por varios componentes:

- **Streamlit** → interfaz donde el usuario hace preguntas
- **RAG Service (rag_service.py)** → gestiona la búsqueda y la generación de respuestas
- **Ingesta (ingest.py)** → procesa los documentos y los guarda en la base vectorial
- **ChromaDB** → almacena los embeddings
- **Gemini** → genera las respuestas

El flujo general del sistema es:



Conclusión

El sistema desarrollado demuestra cómo combinar **embeddings, bases de datos vectoriales y modelos de lenguaje** para construir un sistema de preguntas y respuestas sobre documentos.

Aunque el ejemplo utiliza un PDF pequeño, la misma arquitectura puede ampliarse fácilmente a documentos mucho más grandes. El uso de chunking y búsqueda vectorial permite que el modelo solo utilice los fragmentos relevantes para responder, lo que hace que el sistema sea escalable y eficiente.